

第 30 讲

nn.Module：一个真正的 PyTorch 模型怎样组织 Layer、Parameter 与 Forward
从“零散算子”进入“可训练、可保存、可迁移的模型对象”

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【本讲定位】第 27–29 讲已经学会了 Tensor、nn.Linear、Activation 与 Autograd。现在的问题是：真实模型不可能把几百个 Layer 散落在脚本里。nn.Module 的作用，就是把“有哪些零件、零件里的 Parameter、Forward 数据流、保存/加载、设备迁移”等组织成一个统一模型对象。

学习目标

学完本讲，你应该能够：

1. 解释 nn.Module 是什么、为什么需要它；
2. 区分 `__init__()` 与 `forward()` 的职责；
3. 自己写一个最小 TinyMLP(nn.Module)；
4. 解释为什么推荐 `model(x)` 而不是直接 `model.forward(x)`；
5. 使用 `parameters()` / `named_parameters()` 观察可训练参数；
6. 计算模型总参数量并核对 Shape；
7. 区分 Parameter、普通 Tensor 与 Buffer；
8. 解释 `state_dict()` 保存的是什么、不保存什么；
9. 理解 `model.to(device)` 为什么能一次移动整个模型；
10. 把这些概念重新连接到 Transformer Block 与 LLM 模型结构。

1 【原理】为什么需要 nn.Module？

前面我们可以这样写：

```
fc1 = nn.Linear(3, 4)
act = nn.ReLU()
fc2 = nn.Linear(4, 2)
y = fc2(act(fc1(x)))
```

对于三个 Layer 还可以接受。但真实网络可能有几十层甚至上百层，还需要：

- 找出所有可训练参数；
- 把它们交给 Optimizer；
- 一次移动到 CPU/GPU；
- 保存参数，再加载参数；
- 切换 train/eval 模式；
- 组织嵌套子模块；
- 让 Autograd 与模型结构自然配合。

nn.Module 可以理解成 PyTorch 对“神经网络模型对象”的统一管理协议。它不是一种新的数学运算，而是把 Layer、Parameter、Forward 和模型状态组织起来。

2 【通俗解释】把 nn.Module 想成“机器总装图”

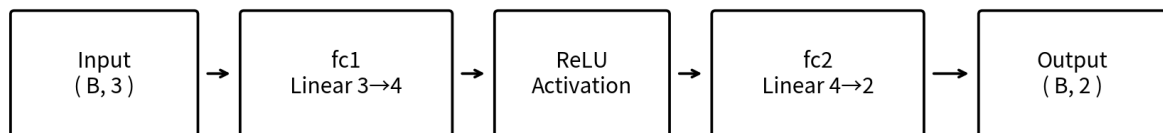
假设一台机器有：第一组齿轮、开关、第二组齿轮。你不希望每次运行都手工去仓库找零件，而是希望有一个“机器对象”：

我有哪些零件？每个零件的参数在哪里？输入从哪里进去？按什么顺序经过零件？最后从哪里出来？

在 PyTorch 里：

- `__init__()`：告诉模型“我有哪些零件”；
- `forward()`：告诉模型“数据按什么顺序经过这些零件”；
- `Parameter`：告诉优化器“哪些数字可以训练”；
- `state_dict()`：告诉保存系统“哪些状态要持久化”。

TinyMLP：`__init__` 注册模块，`forward` 定义数据流



可训练 Parameter：fc1.weight / fc1.bias

可训练 Parameter：fc2.weight / fc2.bias

图 1: TinyMLP 的最小 Module 结构

3 【完整代码骨架】第一个模型类

```
class TinyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(3, 4)
        self.act = nn.ReLU()
        self.fc2 = nn.Linear(4, 2)

    def forward(self, x):
        z1 = self.fc1(x)
        a1 = self.act(z1)
        y = self.fc2(a1)
        return y
```

逐项拆解

代码	真正含义
<code>class TinyMLP(nn.Module)</code>	声明这个类遵循 PyTorch Module 管理协议。
<code>super().__init__()</code>	初始化父类内部的注册机制。省略它会破坏 Module 的正常管理。
<code>self.fc1 = nn.Linear(...)</code>	不只是普通赋值：因为右侧本身是 Module，PyTorch 会把它登记成子模块。
<code>self.act = nn.ReLU()</code>	登记一个没有可训练 Parameter 的激活子模块。
<code>forward(self,x)</code>	定义输入 Tensor 到输出 Tensor 的计算路径。

4 【具体数字例子】 $3 \rightarrow 4 \rightarrow 2$ 的 TinyMLP

输入：

$$X : (B, 3)$$

第一层：

$$W_1 : (4, 3), \quad b_1 : (4,)$$

PyTorch 计算：

$$Z_1 = XW_1^T + b_1 : (B, 4)$$

ReLU 后：

$$A_1 : (B, 4)$$

第二层：

$$W_2 : (2, 4), \quad b_2 : (2,)$$

最后：

$$Y = A_1W_2^T + b_2 : (B, 2)$$

本讲固定参数后，输入：

$$X = [1, 2, -1]$$

手工得到：

$$Z_1 = [-3.4, 2.3, 2.8, -1.4]$$

$$A_1 = [0, 2.3, 2.8, 0]$$

$$Y = [-0.25, 4.75]$$

而 `model(x)` 得到完全相同结果，最大绝对误差为 0。

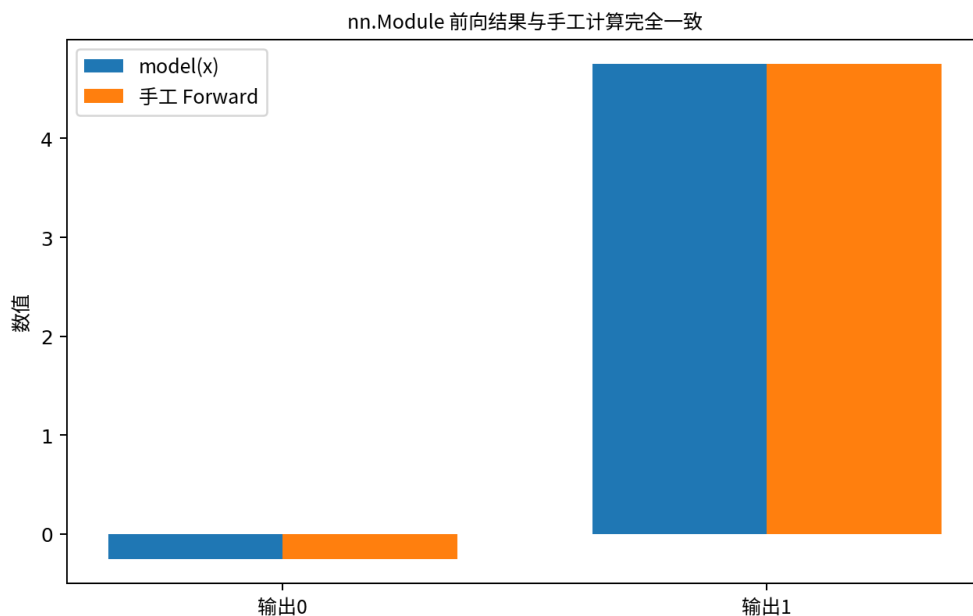


图 2: `model(x)` 与手工 Forward 数值完全一致

5 【Parameter】模型到底认哪些数字是“可训练参数”？

运行：

```
for name, p in model.named_parameters():
    print(name, p.shape, p.numel())
```

得到：

名称	Shape	元素数
fc1.weight	(4, 3)	12
fc1.bias	(4,)	4
fc2.weight	(2, 4)	8
fc2.bias	(2,)	2
合计	—	26

因此：

$$(3 \times 4 + 4) + (4 \times 2 + 2) = 26$$

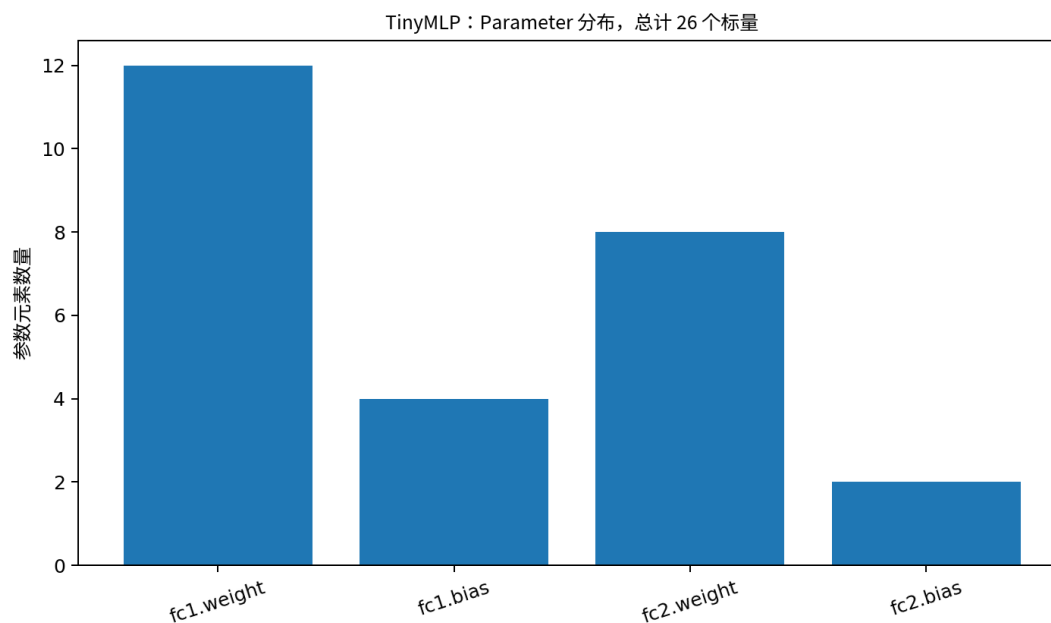


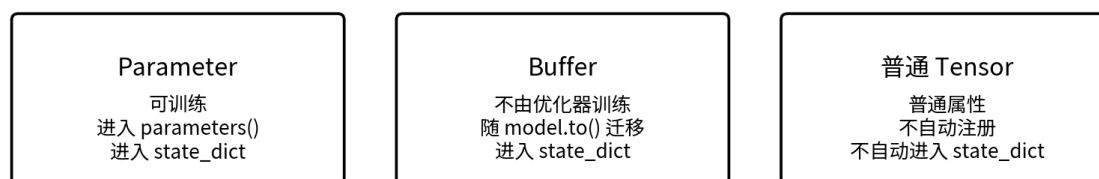
图 3: TinyMLP 中四个 Parameter Tensor 的元素数量

`model.parameters()` 并不是“重新创造参数”，而是在 Module 已登记的参数树中遍历并返回 Parameter。Optimizer 后面正是依赖这套注册机制。

6 【关键区分】Parameter、普通 Tensor、Buffer

这是科研代码中非常重要但初学者经常忽略的区别。

nn.Module 中三类状态的最小区别



`state_dict` = 模型可持久化状态；不是 Python 类定义，也不是 Optimizer 状态

图 4: Module 内部三类状态的最小区别

1. Parameter

```
self.scale = nn.Parameter(torch.tensor(1.5))
```

它通常代表：

- 需要优化器更新；
- 出现在 `named_parameters()`；
- 出现在 `state_dict()`；
- 会随 `model.to(device)` 迁移。

2. 普通 Tensor 属性

```
self.plain_tensor = torch.tensor(2.0)
```

它只是一个 Python 属性。PyTorch 不会因为它“是 Tensor”就自动把它当成模型状态。

3. Buffer

```
self.register_buffer("fixed_offset", torch.tensor(3.0))
```

Buffer 适合：

需要随模型保存、加载、迁移设备，但不是 Optimizer 要训练的状态。

例如后面学习 BatchNorm 时，会看到运行统计量属于典型 Buffer 类状态。

不要把“requires_grad=False 的普通 Tensor”机械理解成 Buffer。Buffer 是显式注册进 Module 状态管理系统的。

7 【state_dict】模型保存的到底是什么？

对于 TinyMLP：

```
model.state_dict().keys()
```

得到：

```
fc1.weight  
fc1.bias  
fc2.weight  
fc2.bias
```

state_dict() 可以理解成“这个 Module 当前可持久化状态的键值表”。它主要包含 Parameter 与持久 Buffer。

它不等于整个 Python 模型代码。如果只保存 `state_dict()`，加载时仍然需要先构造兼容的模型结构：

```
model2 = TinyMLP()
model2.load_state_dict(torch.load(path))
```

实验中，随机新模型加载前输出与原模型不同；加载同一个 `state_dict()` 后，输出与原模型最大绝对误差为 0。

科研复现中，常见最低要求不是只写“用了某个模型”，而是同时明确模型结构、超参数、权重 checkpoint、代码版本与加载方式。仅有 `state_dict` 本身并不能完整说明实验。

8 【`model.to(device)`】为什么能一次移动整个模型？

如果模型中的 `Parameter` / `Buffer` 已经正确注册：

```
model = model.to(device)
```

PyTorch 可以递归遍历整个 `Module` 树，把注册状态迁移到目标设备。

本次容器验证环境为 CPU-only，因此实验得到：

```
Parameter scale.device = cpu
Buffer fixed_offset.device = cpu
```

在正确安装 CUDA 版 PyTorch 的机器上，同一代码可迁移到 GPU。

普通 `Tensor` 属性不会因为“挂在 `self` 上”就自动获得所有 `Module` 管理行为。若它是模型持久状态，应考虑 `Parameter` 或 `register_buffer`，而不是依赖偶然行为。

9 【为什么推荐 `model(x)`，而不是 `model.forward(x)`？】

在这个最小例子里：

$$\text{model}(x) = \text{model.forward}(x)$$

数值上完全一致。

但一般代码应该写：

```
y = model(x)
```

因为真正入口是 `Module` 的 `__call__` 机制，它会处理 `Module` 的一些通用逻辑，然后再调用你定义的 `forward()`。例如 `forward hooks` 就依赖这条调用路径。

你负责写 `forward()`；日常使用时通常调用 `model(x)`。

10 【与 Autograd 的关系】Module 不等于 Backpropagation

`nn.Module` 负责组织模型；Autograd 负责沿实际 Forward 产生的计算图求梯度。

例如：

```
pred = model(x)
loss = pred.sum()
model.zero_grad()
loss.backward()
```

之后：

```
model.fc1.weight.grad
model.fc1.bias.grad
model.fc2.weight.grad
model.fc2.bias.grad
```

都会得到与各自 Parameter 同 Shape 的梯度。

因此完整关系是：

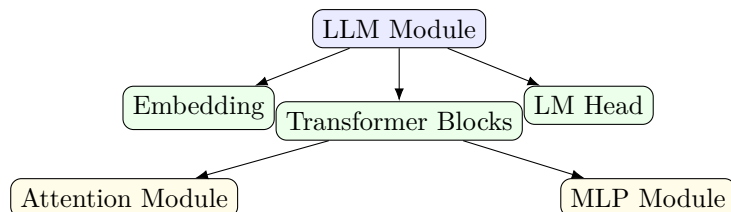
Module 组织 Parameter 和 Forward + Autograd 计算这些 Parameter 的 Gradient

11 【LLM / Transformer 连接】大型模型只是巨大而嵌套的 Module 树

你以前在 LLM 课程中见过：

- Token Embedding；
- Attention；
- Q/K/V Linear；
- MLP / FFN；
- LayerNorm / RMSNorm；
- 多个 Transformer Block。

在 PyTorch 视角，它们往往被组织成层层嵌套的 `nn.Module`：



所以“看懂 LLM PyTorch 源码”的一个最低能力，就是能读懂 Module 树：

这个子模块注册了什么 Parameter？Forward 输入输出 Shape 是什么？`state_dict` 中对应哪些键？

12 【常见误区】

1. **误区：写了 `forward()`，就必须自己调用 `forward()`。** 日常使用通常调用 `model(x)`。
2. **误区：任何挂在 `self` 上的 Tensor 都是 Parameter。** 错。需要显式 Parameter 或由子 Module 注册。
3. **误区：`state_dict()` 就是完整模型。** 错。它是状态，不是 Python 类定义本身。
4. **误区：Parameter 就只是 `requires_grad=True` 的 Tensor。** Parameter 还带有 Module 自动注册语义。
5. **误区：`model.to(device)` 会神奇地移动所有 Python 属性。** 它重点管理已注册 Module/Parameter/Buffer。
6. **误区：有了 Module 就自动训练。** Module 只组织模型；训练还需要 Loss、Backward、Optimizer step 等。

13 【科研阅读方法】看到一个自定义 Module，先读哪几处？

建议固定按照下面顺序：

1. 看 `__init__()`：有哪些 Layer / Parameter / Buffer；
2. 给每个关键 Parameter 写 Shape；
3. 看 `forward()`：Tensor 怎样流动，Shape 怎样变化；
4. 看是否存在条件分支、残差、共享参数；
5. 看模型输出究竟是什么，后续 Loss 怎么接；
6. 必要时打印 `named_parameters()` 与 `state_dict().keys()` 做核对。

对于 Agent Systems 论文，如果涉及训练一个 router、value model、reward model、memory encoder 或小型 policy head，你不一定需要自己发明底层网络，但必须有能力看懂它们的 Module 组织与参数是否真的进入优化过程。否则很容易出现“代码能跑，但某个模块其实没有被注册/更新”的实验错误。

本讲最低闭环

如果现在看到：

```
class MyModel(nn.Module):
    def __init__(self): ...
    def forward(self, x): ...
```

你至少应该立即问：

1. 注册了哪些子 Module？

2. 哪些是可训练 Parameter? Shape 是什么?
3. Forward 中 Tensor 每一步 Shape 怎么变?
4. 最终 Loss 对哪些 Parameter 有梯度路径?
5. 哪些状态会进入 state_dict?

到这里, PyTorch 模型不再只是“一个 class”。它是: 结构 + 参数注册 + Forward 规则 + 状态管理 + Autograd 接口的统一容器。